

Celebal Summer Internship

Week - E-Commerce Sales Database

Part : Database Setup

In []: *#Cell 1*

```
In [12]: # Import DuckDB

import duckdb

# Create Connection

con = duckdb.connect("shopease.db")
```

Create Customers Table

```
In [13]: con.sql("""

CREATE TABLE customers (

customer_id INTEGER PRIMARY KEY,

first_name VARCHAR NOT NULL,

last_name VARCHAR NOT NULL,

email VARCHAR UNIQUE NOT NULL,

city VARCHAR NOT NULL,

state VARCHAR NOT NULL,

join_date DATE NOT NULL,

is_premium BOOLEAN DEFAULT FALSE

);

""")
```

Create Products Table

```
In [14]: con.sql("""

CREATE TABLE products (

product_id INTEGER PRIMARY KEY,

product_name VARCHAR NOT NULL,

category VARCHAR NOT NULL,

brand VARCHAR NOT NULL,
```

```
unit_price DECIMAL(10,2) NOT NULL CHECK(unit_price>0),  
stock_qty INTEGER NOT NULL DEFAULT 0 CHECK(stock_qty>=0)  
);  
""")
```

In []: Create Orders Table

```
In [15]: con.sql("""  
CREATE TABLE orders (  
order_id INTEGER PRIMARY KEY,  
customer_id INTEGER NOT NULL,  
order_date DATE NOT NULL,  
status VARCHAR NOT NULL DEFAULT 'Pending',  
total_amount DECIMAL(12,2) NOT NULL CHECK(total_amount>=0),  
FOREIGN KEY(customer_id)  
REFERENCES customers(customer_id)  
);  
""")
```

In []: Create Order Items Table

```
In [16]: con.sql("""  
CREATE TABLE order_items (  
item_id INTEGER PRIMARY KEY,  
order_id INTEGER NOT NULL,  
product_id INTEGER NOT NULL,  
quantity INTEGER NOT NULL CHECK(quantity>0),  
unit_price DECIMAL(10,2) NOT NULL CHECK(unit_price>0),  
discount_pct DECIMAL(5,2) DEFAULT 0 CHECK(discount_pct BETWEEN 0 AND 100)  
FOREIGN KEY(order_id)  
REFERENCES orders(order_id),  
FOREIGN KEY(product_id)  
REFERENCES products(product_id)  
);  
""")
```

$$\begin{aligned} & \text{)} ; \\ & \text{"" "" "")} \end{aligned}$$

Create Indexes

```
In [17]: con.sql("""

CREATE INDEX idx_customers_city
ON customers(city);

CREATE INDEX idx_customers_state
ON customers(state);

CREATE INDEX idx_products_category
ON products(category);

CREATE INDEX idx_orders_date
ON orders(order_date);

CREATE INDEX idx_orders_status
ON orders(status);

""")
```

```
In [ ]: Insert Sample Data
```

```
In [18]: #customers

con.sql("""

INSERT INTO customers VALUES

(101,'Aarav','Sharma','aarav.s@email.com','Mumbai','Maharashtra','2024-01-15',FAL)

(102,'Priya','Patel','priya.p@email.com','Ahmedabad','Gujarat','2024-02-28',TRU)

(103,'Rohan','Gupta','rohan.g@email.com','Delhi','Delhi','2024-03-10',TRU)

(104,'Sneha','Reddy','sneha.r@email.com','Hyderabad','Telangana','2024-04-05',FAL)

(105,'Vikram','Singh','vikram.s@email.com','Jaipur','Rajasthan','2024-05-20',FAL)

(106,'Ananya','Iyer','ananya.i@email.com','Chennai','Tamil Nadu','2024-06-18',FAL)

(107,'Karan','Mehta','karan.m@email.com','Pune','Maharashtra','2024-07-22',FAL)

(108,'Divya','Nair','divya.n@email.com','Kochi','Kerala','2024-08-30',FAL)

""")
```

```
In [19]: #Products
          con.sql("""
          INSERT INTO products VALUES
```

```
(201,'Wireless Earbuds','Electronics','BoAt',1499,250),
(202,'Cotton T-Shirt','Clothing','Levis',799,500),
(203,'Smart Watch','Electronics','Noise',2999,150),
(204,'Running Shoes','Clothing','Nike',4599,120),
(205,'Bluetooth Speaker','Electronics','JBL',3499,200),
(206,'Bedsheet Set','Home','Spaces',1299,300),
(207,'Laptop Stand','Electronics','AmazonBasics',899,180),
(208,'Cushion Covers (Set)','Home','HomeCenter',599,400);
""")
```

```
In [20]: #Orders
con.sql("""

INSERT INTO orders VALUES

(1001,101,'2024-08-01','Delivered',4498),
(1002,102,'2024-08-03','Delivered',799),
(1003,103,'2024-08-05','Shipped',7498),
(1004,101,'2024-08-10','Delivered',3499),
(1005,104,'2024-08-12','Cancelled',2999),
(1006,105,'2024-08-15','Delivered',5898),
(1007,106,'2024-08-18','Pending',1299),
(1008,103,'2024-08-20','Delivered',899),
(1009,107,'2024-08-25','Shipped',6098),
(1010,108,'2024-08-28','Delivered',1598);

""")
```

```
In [ ]: #Order Items
con.sql("""

INSERT INTO order_items VALUES

(5001,1001,201,2,1499,0),
(5002,1001,207,1,899,10),
(5003,1002,202,1,799,0),
(5004,1003,203,1,2999,0),
(5005,1003,204,1,4599,5),
```

```
(5006,1004,205,1,3499,0),
(5007,1005,203,1,2999,0),
(5008,1006,201,1,1499,10),
(5009,1006,204,1,4599,5),
(5010,1007,206,1,1299,0),
(5011,1008,207,1,899,0),
(5012,1009,205,1,3499,0),
(5013,1009,208,2,599,15),
(5014,1010,206,1,1299,0),
(5015,1010,208,1,599,0);

""" )
```

```
In [21]: #Verify Data
con.sql("SELECT * FROM customers").df()
```

```
Out[21]:
```

	customer_id	first_name	last_name	email	city	state
0		Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra
1		Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat
2		Rohan	Gupta	rohan.g@email.com	Delhi	Delhi
3		Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana
4		Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan
5		Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu
6		Karan	Mehta	karan.m@email.com	Pune	Maharashtra
7		Divya	Nair	divya.n@email.com	Kochi	Kerala

```
In [22]: con.sql("SELECT * FROM products").df()
```

Out[22]:

product_id	product_name	category	brand	unit_price	stock_qty
1	Wireless Earbuds	Electronics	BoAt	1999.0	250
2	Cotton T-Shirt	Clothing	Levis	1999.0	250
3	Smart Watch	Electronics	Noise	1999.0	250
4	Running Shoes	Clothing	Nike	1999.0	250
5	Bluetooth Speaker	Electronics	JBL	1999.0	250
6	Bedsheet Set	Home	Spaces	1999.0	250
7	Laptop Stand	Electronics	AmazonBasics	1999.0	250
8	Cushion Covers (Set)	Home	HomeCenter	1999.0	250

In [23]: `con.sql("SELECT * FROM orders").df()`

Out[23]:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-12	Delivered	1999.0
1002	102	2024-08-12	Delivered	1999.0
1003	103	2024-08-12	Shipped	1999.0
1004	104	2024-08-12	Delivered	1999.0
1005	105	2024-08-12	Cancelled	1999.0
1006	106	2024-08-12	Delivered	1999.0
1007	107	2024-08-12	Pending	1999.0
1008	108	2024-08-12	Delivered	1999.0
1009	109	2024-08-12	Shipped	1999.0
1010	110	2024-08-12	Delivered	1999.0

In [24]: `con.sql("SELECT * FROM order_items").df()`

Out[24]:

item_id	order_id	product_id	quantity	unit_price	discount_pct
---------	----------	------------	----------	------------	--------------

Section A — SQL Basics (Q –Q)

In [25]: `#Display all columns and rows from customers table`
`con.sql("""`
`SELECT * FROM customers;`
`""").df()`

Out[25]:

	customer_id	first_name	last_name	email	city	state
0		Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra
1		Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat
2		Rohan	Gupta	rohan.g@email.com	Delhi	Delhi
3		Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana
4		Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan
5		Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu
6		Karan	Mehta	karan.m@email.com	Pune	Maharashtra
7		Divya	Nair	divya.n@email.com	Kochi	Kerala

In [26]: `#Retrieve first_name, last_name, city`
`con.sql("""`
`SELECT first_name, last_name, city`
`FROM customers;`
`""").df()`

Out[26]:

	first_name	last_name	city
0	Aarav	Sharma	Mumbai
1	Priya	Patel	Ahmedabad
2	Rohan	Gupta	Delhi
3	Sneha	Reddy	Hyderabad
4	Vikram	Singh	Jaipur
5	Ananya	Iyer	Chennai
6	Karan	Mehta	Pune
7	Divya	Nair	Kochi

In [27]: `#List all unique product categories`
`con.sql("""`
`SELECT DISTINCT category`
`FROM products;`
`""").df()`

Out[27]:

category
Electronics
Home
Clothing

Identify Primary Keys

In []: `### Q4 Answer:`

Primary Keys:

- customers → customer_id
- products → product_id
- orders → order_id
- order_items → item_id

Why Primary Key must be UNIQUE and NOT NULL:

- Ensures each record is uniquely identifiable
- Prevents duplicate rows
- Helps maintain data integrity
- Used for relationships (Foreign Keys)

In []: Email constraint explanation

In []: *### Q5 Answer:*

Constraint on email column:

- UNIQUE
- NOT NULL

What happens if duplicate email is inserted?

- Database will throw an ERROR
- Because UNIQUE constraint prevents duplicate values
- Ensures every customer has a unique identity

Insert invalid product (unit_price = -)

```
In [29]: con.sql("""
INSERT INTO products VALUES
(999, 'Invalid Product', 'Test', 'TestBrand', 50, 10);
""")
```

Q Answer:

If we try to insert unit_price = - :

Error occurs because:

CHECK constraint (unit_price >) is violated.

Database prevents invalid data entry to maintain data integrity.

Section B — Filtering & Optimization (Q –Q)

In []: *#Retrieve all orders with status = 'Delivered'*

```
In [30]: con.sql("""
SELECT *
FROM orders
WHERE status = 'Delivered';
""").df()
```


Out[30]:

order_id	customer_id	order_date	status	total_amount
1000	100	2024-01-10	Delivered	.
1001	101	2024-01-24	Delivered	.
1010	104	2024-01-24	Delivered	.
			Delivered	.
			Delivered	.
			Delivered	.
			Delivered	.

Electronics products with price >

In [31]:

```
con.sql("""
SELECT *
FROM products
WHERE category = 'Electronics'
AND unit_price > 2000;
""").df()
```

Out[31]:

product_id	product_name	category	brand	unit_price	stock_qty
	Smart Watch	Electronics	Noise	.	
	Bluetooth Speaker	Electronics	JBL	.	

In [32]:

```
#Customers from Maharashtra who joined in 2024
con.sql("""
SELECT *
FROM customers
WHERE state = 'Maharashtra'
AND join_date BETWEEN '2024-01-01' AND '2024-12-31';
""").df()
```

Out[32]:

customer_id	first_name	last_name	email	city	state
	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra
	Karan	Mehta	karan.m@email.com	Pune	Maharashtra

Orders between dates and NOT cancelled

Explain index idx_orders_date (THEORY)

In []: *### Q11 Answer:*

Index: idx_orders_date

Purpose:

- Speeds up queries filtering by order_date
- Avoids full table scan

Example query benefiting from index:

```
SELECT *
```

```
FROM orders
WHERE order_date = '2024-08-10';
```

Why faster?

- Index allows direct lookup instead of scanning all rows

Q Answer:

Query given: SELECT FROM customers WHERE YEAR(join_date) = ;

Problem:

- Function on column (YEAR()) prevents index usage
- Not SARGable

Optimized query:

```
SELECT                      FROM customers WHERE join_date BETWEEN '                      ' AND '                      ';
```

Why better:

- Index on join_date can be used
- Faster execution

Part : Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

```
In [33]: #Count total number of orders
con.sql("""
SELECT COUNT(*) AS total_orders
FROM orders;
""").df()
```

Out[33]: **total_orders**

Total revenue from Delivered orders

```
In [34]: con.sql("""
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'Delivered';
""").df()
```

Out[34]: **total_revenue**

In []: Average unit price per category

```
In [35]: con.sql("""
SELECT
```

```

        category,
        AVG(unit_price) AS avg_price
FROM products
GROUP BY category;
""").df()

```

Out[35]:

category	avg_price
Electronics	.
Home	.
Clothing	.
Test	.

Order status wise count + revenue

In [36]:

```

con.sql("""
SELECT
    status,
    COUNT(*) AS total_orders,
    SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
""").df()

```

Out[36]:

status	total_orders	total_revenue
Delivered	.	.
Shipped	.	.
Cancelled	.	.
Pending	.	.

In []: Most expensive & cheapest product per category

In [37]:

```

con.sql("""
SELECT
    category,
    MAX(unit_price) AS max_price,
    MIN(unit_price) AS min_price
FROM products
GROUP BY category;
""").df()

```

Out[37]:

category	max_price	min_price
Electronics	.	.
Home	.	.
Test	.	.
Clothing	.	.

In []: Categories **with** avg price > 2000

```
In [38]: con.sql("""
SELECT
    category,
    AVG(unit_price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
""").df()
```

Out[38]:

category	avg_price
Clothing	.
Electronics	.

In []: Q19-Q23 (Joins & Relationships)

```
In [39]: con.sql("""
SELECT
    o.order_id,
    o.order_date,
    c.first_name,
    c.last_name,
    o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
""").df()
```

Out[39]:

order_id	order_date	first_name	last_name	total_amount
1004	2021-09-12	Aarav	Sharma	.
1005	2021-09-18	Priya	Patel	.
1007	2021-09-18	Rohan	Gupta	.
1008	2021-09-28	Sneha	Reddy	.
1010	2021-09-29	Vikram	Singh	.
1001	2021-09-01	Ananya	Iyer	.
1003	2021-09-01	Karan	Mehta	.
		Divya	Nair	.
		Aarav	Sharma	.
		Rohan	Gupta	.

LEFT JOIN

```
In [40]: con.sql("""
SELECT
    c.customer_id,
    c.first_name,
```

```

        c.last_name,
        o.order_id,
        o.order_date,
        o.total_amount
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
""").df()

```

Out[40]:

	customer_id	first_name	last_name	order_id	order_date	total_amount
1	140	Aarav	Sharma	1000	2024-08-12	2000.0
4	140	Priya	Patel	1003	2024-08-16	8000.0
5	140	Rohan	Gupta	1007	2024-08-16	12000.0
6	140	Sneha	Reddy	1009	2024-08-25	8400.0
7	140	Vikram	Singh	1010	2024-08-26	10000.0
8	140	Ananya	Iyer	1011	2024-08-31	8400.0
9	140	Karan	Mehta	1002	2024-08-08	10000.0
		Divya	Nair			
		Aarav	Sharma			
		Rohan	Gupta			

In []: 3 Table JOIN (Orders → Order Items → Products)

In [41]:

```

con.sql("""
SELECT
    o.order_id,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    oi.discount_pct
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
JOIN products p
    ON oi.product_id = p.product_id;
""").df()

```

Out[41]:

order_id	product_name	quantity	unit_price	discount_pct
----------	--------------	----------	------------	--------------

In []: LEFT JOIN vs RIGHT JOIN

In []: *### Q22. Difference between LEFT JOIN and RIGHT JOIN*

- ****LEFT JOIN****: Returns all records **from** the left table **and** matched records
 - ****RIGHT JOIN****: Returns all records **from** the right table **and** matched records
- ### Example from this schema:*
- **LEFT JOIN**:
customers LEFT JOIN orders → all customers shown even **if** they have no orders

- RIGHT JOIN:
orders RIGHT JOIN customers → same result but written **in** reverse.

When to use FULL OUTER JOIN:

Use FULL OUTER JOIN when you want:

- All customers (even without orders)
- AND all orders (even **if** customer data **is** missing)

It returns everything **from** both tables.

In []: Foreign Keys + Invalid Insert Scenario

In []: ### Q23. Foreign Keys Explanation

Foreign Key Relationships:

- orders.customer_id → customers.customer_id
- order_items.order_id → orders.order_id
- order_items.product_id → products.product_id

What happens if we insert:

customer_id = 999 **in** orders?

It will FAIL because:

- 999 does **not** exist **in** customers table
- Foreign key constraint enforces referential integrity

This prevents orphan records **and** keeps data consistent.

In []: Part 5: Advanced SQL (Q24–Q27)

```
In [42]: con.sql("""
SELECT
    product_name,
    unit_price,
    CASE
        WHEN unit_price < 1000 THEN 'Budget'
        WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
        ELSE 'Premium'
    END AS price_tier
FROM products;
""").df()
```

```
Out[42]:
```

product_name	unit_price	price_tier
Wireless Earbuds	.	Mid-Range
Cotton T-Shirt	.	Budget
Smart Watch	.	Mid-Range
Running Shoes	.	Premium
Bluetooth Speaker	.	Premium
Bedsheet Set	.	Mid-Range
Laptop Stand	.	Budget
Cushion Covers (Set)	.	Budget
Invalid Product	.	Budget

```
In [ ]: CASE inside Aggregation (Delivered vs Not Delivered)
```

```
In [43]: con.sql("""
SELECT
    COUNT(CASE WHEN status = 'Delivered' THEN 1 END) AS delivered_orders,
    COUNT(CASE WHEN status != 'Delivered' THEN 1 END) AS not_delivered_or
FROM orders;
""").df()
```

```
Out[43]:
```

delivered_orders	not_delivered_orders
------------------	----------------------

```
In [ ]: ACID Properties
```

```
In [ ]: ### Q26. ACID Properties Explained
```

****A - Atomicity****

A transaction **is** fully completed **or** fully failed.

Example: Bank transfer – money **is** debited **and** credited together.

****C - Consistency****

Database remains valid before **and** after transaction.

Example: Account balance cannot become negative **if not** allowed.

****I - Isolation****

Multiple transactions do **not** interfere **with** each other.

Example: Two users booking last seat simultaneously.

****D - Durability****

Once transaction **is** committed, it **is** permanent even after crash.

Example: Successful payment remains saved even **if** system crashes.

Transaction